EVENT MANAGEMENT

Presented by :
Pooja & Neha

problem Statement

Problem

- Managing multiple events with dependencies is difficult
- Events may rely on completion of other events
- Incorrect planning can lead to circular dependencies
- Manual tracking is error-prone

Statement

- Design and implement a system to:
  - Create and manage events
  - Define dependencies between events
  - Detect and prevent cyclic dependencies
  - Allow updating and deleting events efficiently

Real -World Application

01 Project Planning (task scheduling)

02 Software build system(module dependencies)

03 College event management

04 Workflow system(approval chains)

Example:
“Venue Booking → Decoration → Event Start”

Data Structure Used

- *Graph (Directed Graph)*
  Nodes → Events
  Edges → Dependencies
- *Adjacency Matrix*
  adj[i][j] = 1 → Event i depends on Event j

Structure Used:

```
struct Event {
  int id;
  char name[100];
};
```

System Design

Modules:

1. Create Event
2. Add Dependency
3. Display Events
4. Update Event
5. Delete Event
6. Cycle Detection

Flow :

- User inputs →System processes → Graph updated
- Cycle check ensures valid structure

Algorithm

Cycle Detection (DFS Approach)

1. Mark node as visited
2. Add to recursion stack
3. Traverse neighbors
4. If node already in recursion stack → Cycle exists

Steps:

- Use visited[] array
- Use recStack[] array
- Recursively check all nodes

Key Function

- createEvent() → **Adds new event**
- addDependency() → **Adds edge in graph**
- hasCycle() → **Detects cycle**
- display() → **Shows events & dependencies**
- updateEvent**()** → **Edits event name**
- deleteEvent() → **Removes event & updates graph**

Implimentation Details

- Language: C
- Max events: 50
- Unique ID for each event
- Uses:
  - Arrays
  - Structures
  - DFS recursion

Demo Output

Example: **Sample Output 1 – Basic Creation**

```
--- Event Management System ---
1. Create Event
2. Add Dependency
3. Display Events & Dependencies
4. Update Event
5. Delete Event
6. Exit
Enter choice:
```

```
Enter choice: 1
Enter event name: Marriage
Event created!
```

```
Enter choice: 1
Enter event name: Birthday Party
Event created!
```

```
Enter choice: 1
Enter event name: Enagagement
Event created!
```

Sample Output 2 – Adding Dependencies

```
Enter choice: 2
Enter prerequisite event ID: 1
Enter dependent event ID: 2
Dependency added!
```

```
Enter choice: 2
Enter prerequisite event ID: 2
Enter dependent event ID: 3
Dependency added!
```

Sample Output 3 – Display Events & DAG

```
Enter choice: 3

Events:
[1] Marriage
[2] Birthday Party
[3] ngagement
[4] Enagagement

Dependencies:
Marriage -> Birthday Party
Birthday Party -> ngagement
```

Sample Output 4 – Cycle Detection

```
Enter choice: 2
Enter prerequisite event ID: 3
Enter dependent event ID: 1
Cycle detected! Dependency not added.
```

Sample Output 5 – Update Event

```
Enter choice: 4
Enter event ID to update: 2
Enter new name: Reception
Event updated!
```

Sample Output 6 – Delete Event

```
Enter choice: 5
Enter event ID to delete: 3
Event deleted!
```

Sample Output 7 – Final Display

```
Enter choice: 3

Events:
[1] Marriage
[2] Reception
[4] Enagagement

Dependencies:
Marriage -> Reception
```

Sample Output 8 – Exit

```
--- Event Management System ---
1. Create Event
2. Add Dependency
3. Display Events & Dependencies
4. Update Event
5. Delete Event
6. Exit
Enter choice: 6
Exiting...
```

Conclusion

- Successfully implemented event dependency system
- Prevents logical errors using cycle detection
- Demonstrates real use of graphs in C
- Can be extended with:
  - GUI interface
  - File storage
  - Topological sorting

Thank You.